Multi-Agent Financial Analyst: Investment Research Agent

GitHub repository: https://github.com/sudhakaran-srinivasan/multi-agent-financial-analyst

Group 6: Sudhakaran Srinivasan, April Bradick, Jeffery Smith. AAI-520 final team project.

Submitted version: git tag v1.0-group6-submission

This notebook demonstrates an autonomous investment research agent for one stock at a time, built from scratch in Python (`src/finagent/`). It runs the whole system end to end, then each module on its own, then the safety checks and the evaluator loop. Run cells top to bottom the first time.

Part	What it is	Needs API keys?
Design	Agent design and workflows (read first)	no
0	Setup	yes (OpenRouter, Finnhub in `.env`)
1	The overall system, end to end	yes
2	Each module on its own (agent functions and the news workflow)	mostly yes; the last cells are offline
3	Evaluation and iteration, safety checks, known limits	offline unless noted

Cost note: Part 1 with the evaluator makes roughly 10 to 45 model calls per ticker (the news step is the biggest share). Finnhub free tier is 60 calls/minute, so avoid re-running Part 1 in a tight loop.

Educational project, not investment advice.

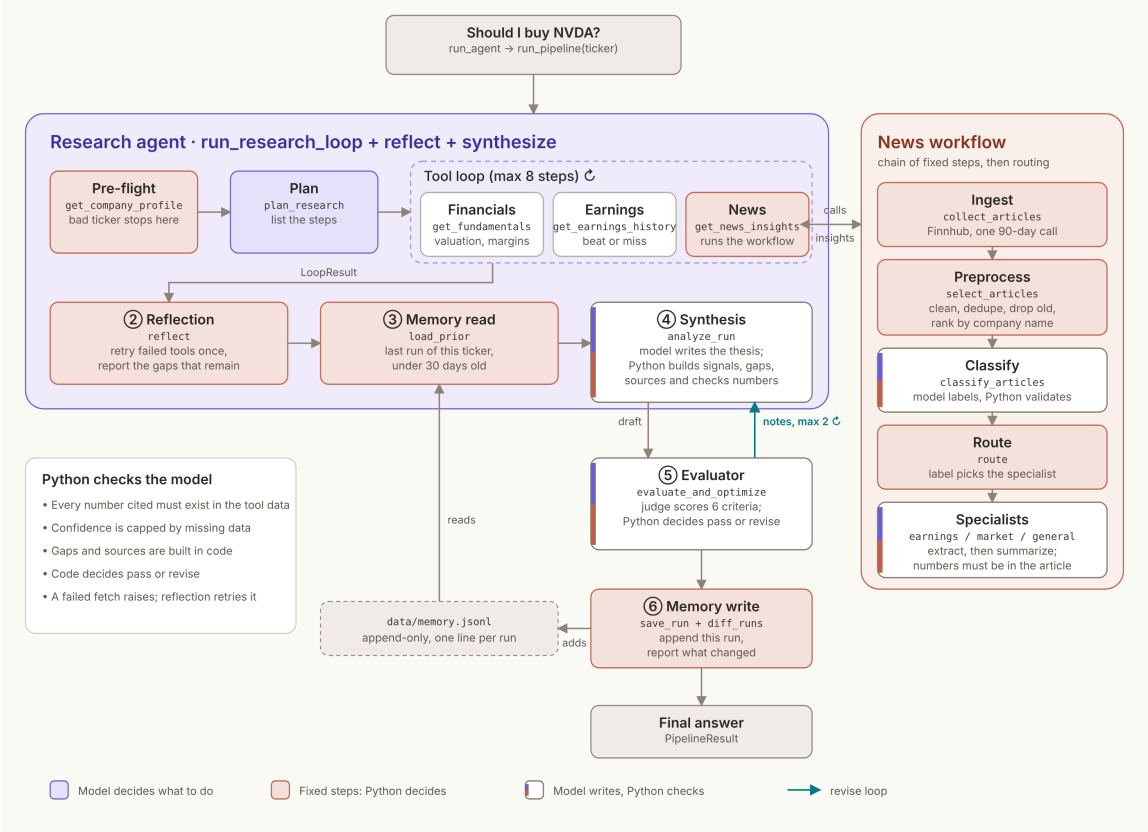
Reviewer's guide

Rubric item	Where it is in this notebook
Agent functions: planning, dynamic tool use, self-reflection, learns across runs	**Agent Functions and Capabilities** (Parts 0 to 2): 1a, 2.4 (plan and tool use), 2.5 (reflection), 2.9 and 2.10 (memory), recap table
Workflow patterns: prompt chaining, routing, evaluator-optimizer	Design table; 2.7 (prompt chaining and routing, station by station); **Evaluation and Iteration** (Part 3) and 1a (evaluator, live)

Rubric item	Where it is in this notebook
Code: readable notebook, GitHub link, comments on design, functions and evaluation	GitHub link above; the three sections **Agent Design and Workflows**, **Agent Functions and Capabilities** and **Evaluation and Iteration**; summary table at the end

To reproduce (full steps in the README): clone the repository, use Python 3.11 or newer, run `python -m pip install -e ".[dev]"`, copy `.env.example` to `.env` and add `OPENROUTER_API_KEY` and `FINNHUB_API_KEY`, then run `python -m finagent.run_agent NVDA --analyze --evaluate` or this notebook. Live data and model output vary from run to run; the outputs below are from one run.

Agent Design and Workflows



`run_pipeline` chains six stages: **research loop** (pre-flight profile, plan, tool calls) → **reflection** → **memory read** → **synthesis** → **evaluator-optimizer** → **memory write**. A crash in a later stage is recorded in `stage_errors` and the run returns what it has.

Who decides what

- **The model** chooses which tools to call, writes the thesis, labels and summarizes news, and scores drafts.
- **Python** validates every tool call, computes the signals, builds data gaps and sources, checks every evidence number, caps confidence, retries failed tools, and

decides pass or revise.

The three workflow patterns

Pattern	Design	Code
Prompt chaining	News is a fixed chain: Ingest (Finnhub, one 90-day call) → Preprocess (clean, dedupe, drop old, rank by company name) → Classify (model labels, Python validates) → Extract (facts) → Summarize (summary and sentiment). Every number in a summary must appear in the article.	`workflows/news_ingest.py`, `news_classify.py`, `news_specialists.py`
Routing	The classifier's label (earnings, market, general, irrelevant) selects one of three specialist prompts. A bad label is skipped and counted, never guessed.	`route` in `news_specialists.py`
Evaluator-optimizer	Draft → judge scores six criteria → Python decides → revise with feedback, at most 2 rounds. See "Evaluation and iteration: design comments" in Part 3.	`workflows/evaluator_optimizer.py`

Why it is built this way

- **Fail loudly:** a failed fetch raises so the trace shows `ok=False` and reflection can retry. An empty list would look like success.
- **Pre-flight check:** a bad ticker or an ETF stops the run before any model call is spent.
- **Everything that calls out is injectable**, so the whole test suite runs with no model and no network.

Full diagrams, naming map and decision log: docs/ARCHITECTURE.md.

Data sources and services

Source	Used for	Key needed
Yahoo Finance via `yfinance`	company profile, price, valuation, margins, growth, balance sheet	no
Finnhub	company news (one 90-day call per run) and earnings history	`FINNHUB_API_KEY` (free tier, 60 calls/minute)

Source	Used for	Key needed
OpenRouter	the language model for the planner, tool loop, news steps, synthesis and judge (default `openai/gpt-4o-mini`)	`OPENROUTER_API_KEY`

NewsAPI, Alpha Vantage, FRED and SEC EDGAR were considered but not used. Their free tiers are tight for a loop that may retry (Alpha Vantage about 25 calls a day, NewsAPI about 100 a day), and Yahoo Finance plus Finnhub cover the profile, fundamentals, earnings and news this agent needs.

Key design decisions

Decision	Why	Code
The model proposes, Python checks	A model can omit a gap, round a number into a different one, or sound more certain than the data allows.	`planner.py`, `loop.py` (model); `reflection.py`, `evaluation_rules.py` (code)
Pre-flight profile before planning	A bad ticker or an ETF stops the run before any model call is spent; the profile also shapes the plan.	`get_company_profile`
Reflection is code, not a model	A retry helps transient failures only; a permanent gap (negative earnings) must not be retried.	`reflect`
Confidence has a Python ceiling	Missing fundamentals or earnings history cap confidence; the model may ask for less, never more.	`confidence_ceiling`, `apply_ceiling`
Evidence must be verifiable	Each item's source path must exist and its number match within 2%, or it is dropped.	`check_evidence`
The judge scores, Python decides	Pass needs mean at least 4.0, no score below 3, no critical issue. A rule in code cannot be talked into a pass.	`decide_verdict`
Memory is append-only, filtered on read	Nothing is deleted; only the newest record under 30 days is used.	`memory.py`

The full decision log and naming map are in docs/ARCHITECTURE.md.

Agent Functions and Capabilities

Parts 0 to 2 run the agent end to end (Part 1), then each capability on its own (Part 2): planning, dynamic tool use, self-reflection and memory, plus the news workflow (prompt

chaining and routing) that the agent calls as a tool.

0. Setup

In [1]:
```
import os, sys
from pathlib import Path

# Run from the repo root so `.env` and `data/` are found.
root = Path.cwd()
while root != root.parent and not (root / "pyproject.toml").exists() and
not (root / "src").exists():
    root = root.parent
os.chdir(root)
sys.path.insert(0, str(root / "src"))
print("repo root:", root.name)

from finagent import config
config.load_environment()
for name in (config.OPENROUTER_KEY_NAME, config.FINNHUB_KEY_NAME):
    print(f"{name}:", "set" if os.environ.get(name) else "MISSING (add it
to .env)")
```

```
repo root: multi-agent-financial-analyst
OPENROUTER_API_KEY: set
FINNHUB_API_KEY: set
```

1. The overall system

`run_pipeline` is the single function that chains every stage:

`profile -> plan -> tool loop -> reflection -> [memory read] -> synthesis -> [evaluator-optimizer] -> [memory write]`

1a. A healthy mega-cap, with the evaluator on

Things to look for: the **Plan** (why these tools), the **Trace** (which tools ran), the **Scorecard** (computed by Python, not the model), the **Evaluation** block (scores per criterion, revisions), and whether **Risks** are tied to real data.

In [2]:
```
from finagent.pipeline import run_pipeline
from finagent.run_agent import format_report, format_analysis
from finagent.workflows.evaluator_optimizer import evaluate_run

run = run_pipeline("NVDA", analyze=True, evaluator=evaluate_run)  # memory
off here; see 2.9
print(format_report(run.loop))
print(format_analysis(run.outcome))
```

```
NVDA  |  stop: done  |  model steps: 4
Profile: NVIDIA Corporation (EQUITY), mega-cap, profitable, earnings in 30 d
ays

Plan:
  1. get_fundamentals: As a mega-cap technology company, understanding NVIDI
A's key financial ratios will provide insights into its valuation, profitabi
lity, and growth potential.
  2. get_earnings_history: Analyzing the last four quarters of earnings vers
us analyst estimates is crucial for assessing NVIDIA's performance consisten
cy and market expectations ahead of the upcoming earnings report.
  3. get_news_insights: Recent news insights will help gauge market sentimen
t and any developments that could impact NVIDIA's stock performance leading
up to its earnings date.
  Emphasis: This plan emphasizes a comprehensive financial and sentiment ana
lysis to prepare for NVIDIA's upcoming earnings report.

Trace (step 0 = pre-flight check):
   0  get_company_profile    ok
   0  plan_research          ok
   1  get_fundamentals       ok
   2  get_earnings_history   ok
   3  get_news_insights      ok

Results collected:
  get_fundamentals       gaps: 0   warnings: 0
  get_earnings_history   gaps: 0   warnings: 0
  get_news_insights      8 items

News insights (category, sentiment, summary):
  2026-10-19  general  positive A $10,000 investment in Nvidia stock is proj
ected to increase in value by 2030, driven by emerging growth oppo...
  2026-10-18  market   negative Bond investors are showing less confidence i
n Nvidia, which serves as a warning to Nvidia shareholders.
  2026-10-18  general  positive Nvidia is launching a significant financing
initiative, highlighting that the AI race now relies heavily on ca...
  2026-10-18  general  positive Honeywell has introduced a new platform call
ed Forge Cognition, which is backed by Nvidia.
  2026-10-18  market   positive Nvidia stock has historically produced an av
erage return of 13% in November this decade, although there is one...
  2026-10-18  market   neutral  NVIDIA Corporation closed at $217.23 on Octo
ber 15 and has returned 9.52% over the last six months, while also...
  2026-10-18  general  positive Elon Musk praised Nvidia CEO Jensen Huang as
SpaceX and Nvidia collaborate on AI computers for space, aiming t...
  2026-10-18  general  positive Nvidia announced its largest share repurchas
e authorization in market history and is valued at $5 trillion.

ANALYSIS  |  stance: bullish  |  confidence: high

NVIDIA Corporation demonstrates strong financial health with a solid net cas
h position of $23.61 billion, indicating that cash exceeds debt. The company
has consistently beaten earnings estimates, achieving a perfect beat rate of
100% over the last four quarters, which reflects its strong operational perf
ormance. Additionally, positive signals such as a healthy profit margin of 6
3.66% and a revenue growth rate of 5.9% further support the bullish outlook.
Analyst sentiment is also favorable, with a target mean price suggesting a p
```

otential upside of over 54% from the current price.

Scorecard (computed by Python; [+] positive, [=] neutral, [-] negative):

Signal	Value	Read	Why
forward_vs_trailing_pe	13.45	[+]	forward P/E 13.5 is below trailing 27.1: analysts expect earnings to grow
profit_margin	0.6366	[+]	net profit margin is healthy (10% or more)
revenue_growth	1.059	[+]	sales are growing 10% a year or faster
net_cash	23.6B	[+]	cash exceeds debt (a net cash position)
fcf_yield	0.02456	[=]	free cash flow is positive but under 3% of market value
liquidity	4.589	[+]	short-term assets comfortably cover short-term bills
trend_vs_200d	214.1	[+]	price 214.13 is above its 200-day average 202.97 (the long-term trend)
analyst_target	328.7	[+]	analyst average target 328.72 is +54% from the price (59 analysts; an opinion, not a fact)
earnings_record	1	[=]	beat estimates in 4 of 4 quarters; beating estimates is the norm (about 78% of S&P 500 companies), so this is not a strength

Evidence:
  - net cash position = 23608999936.0 <get_fundamentals.derived.net_cash>
  - profit margin = 0.63663 <get_fundamentals.profitability.profitMargins>
  - revenue growth = 1.059 <get_fundamentals.growth.revenueGrowth>
  - analyst target mean price = 328.71695 <get_fundamentals.sentiment.targetMeanPrice>
  - earnings beat rate = 1.0 <get_earnings_history.summary.beat_rate>

Risks:
  - Bond investors are showing less confidence in Nvidia, which serves as a warning to Nvidia shareholders (source: get_news_insights).

Data gaps:
  (none)

Sources:
  - Yahoo Finance via yfinance (fundamentals)
  - Finnhub (earnings history)
  - https://finnhub.io/api/news?id=32a15317c6b93e3bcc0eb16ccf6d38930ed253f8283e8ee819529d513fde0167
  - https://finnhub.io/api/news?id=4a3790b182ec464e7975e4d12063aab3ad911fa97a074e47cc5c30ef97ee15be
  - https://finnhub.io/api/news?id=169d402f14651d765be2dbf79995fb18684bf49db53d994147fe7d15c9f096ac
  - https://finnhub.io/api/news?id=5ac22a9eb0829bd8df3a9e3ecbfbf82058f4e386211559a1961f91ecb6ad786e
  - https://finnhub.io/api/news?id=f700f4c46ecd5d3e9406b653eab38013434f79f9a0319f698e7f0366c5bc13a2
  - https://finnhub.io/api/news?id=8fceddf7e2520bda5a211bc2dd69e28da0298261de986f6370a0e444ece7e5a7
  - https://finnhub.io/api/news?id=cc764e9f0d815b42034cff066b749aaa04c6b81c2

```
b3f140fd9997085a91c0e2e
  - https://finnhub.io/api/news?id=6d7070d4d1c8c5e84c905facedfc79308aeb0e0dd
21365e1096510135cd4dc54

Educational analysis produced by a course project. Not investment advice.
```

In [3]:
```python
# The evaluator's work, round by round.
ev = run.evaluation
print("passed:", ev.passed, "| revisions:", ev.revisions, "|",
ev.stopped_because)
for r in ev.rounds:
    print(f"\n{r.label}: verdict={r.verdict} overall={r.overall:.2f}")
    for s in r.judgment.scores:
        print(f"  {s['criterion']:<24} {s['score']}  {s['comment']}")
    for c in r.judgment.critical_issues:
        print("  CRITICAL:", c)
```

```
passed: True | revisions: 0 | the draft passed

draft: verdict=pass overall=4.33
  evidence_support         5  Every claim in the thesis is backed by specifi
c evidence from the data.
  numeric_accuracy         5  All numbers in the analysis match the raw tool
results exactly.
  stance_consistency       5  The bullish stance is supported by the positiv
e evidence and signals presented.
  risk_coverage            3  Some risks are mentioned, but the analysis lac
ks a comprehensive discussion of potential risks and data gaps.
  confidence_calibration   3  The high confidence level is somewhat unsuppor
ted given the mixed signals from the news insights.
  source_quality           5  All evidence used in the analysis has a named
source.
```

Reading the evaluator output above. Each round shows the verdict, the overall score, one score and comment per criterion, and any critical issues. `revisions` is how many times the draft was sent back; `stopped_because` says why the loop ended (pass, revision cap, or a judge/reviser failure).

1b. Same system, different situations

Each ticker tests a different path. Run them one at a time.

Ticker	Why it is interesting
`AADX`	small-cap, **loses money**: P/E is not meaningful, so signals and gaps differ
`ZZZZZZ`	**not a real ticker**: a fatal error that should stop cleanly, no crash
`SPY`	**an ETF, not a stock**: the pre-flight check should refuse it

In [4]:
```python
run_aadx = run_pipeline("AADX", analyze=True, evaluator=evaluate_run)
print(format_analysis(run_aadx.outcome))
print("\nstage errors:", run_aadx.stage_errors)
```

ANALYSIS | stance: neutral | confidence: medium

Applied Aerospace & Defense, Inc. (AADX) is currently facing challenges due to its unprofitability and significant cash burn, as indicated by a negative profit margin of -0.30236998 and a free cash flow yield of -0.06602931254360155. However, the company shows promising revenue growth of 0.474, suggesting potential for future profitability. Analyst sentiment remains cautiously optimistic with a target mean price of 22.875, indicating a potential upside from the current price of 11.37. The mixed signals regarding profitability and growth lead to a neutral stance on the stock.

Scorecard (computed by Python; [+] positive, [=] neutral, [-] negative):

Signal	Value	Read	Why
forward_vs_trailing_pe	28.6	[=]	trailing earnings are negative; analysts expect a profit at 28.6x forward earnings (a forecast, not a result)
profit_margin	-0.3024	[-]	the company loses money on each dollar of sales
revenue_growth	0.474	[+]	sales are growing 10% a year or faster
net_cash	-413.4M	[-]	debt exceeds cash (net debt)
fcf_yield	-0.06603	[-]	the business burns cash after capital spending
liquidity	2.509	[+]	short-term assets comfortably cover short-term bills
trend_vs_200d	11.37	[-]	price 11.37 is below its 200-day average 16.48 (the long-term trend)
analyst_target	22.88	[+]	analyst average target 22.88 is +101% from the price (8 analysts; an opinion, not a fact)
earnings_record	0	[=]	only 1 quarter(s) of history (0 beat, 1 miss): too thin to draw a conclusion

Evidence:
- negative profit margin = -0.30236998 <get_fundamentals.profitability.profitMargins>
- revenue growth = 0.474 <get_fundamentals.growth.revenueGrowth>
- current price = 11.37 <get_fundamentals.price.currentPrice>
- analyst target mean price = 22.875 <get_fundamentals.sentiment.targetMeanPrice>
- free cash flow yield = -0.06602931254360155 <get_fundamentals.derived.fcf_yield>

Risks:
- negative profit margin from get_fundamentals.profitability.profitMargins
- negative free cash flow yield from get_fundamentals.derived.fcf_yield
- net cash position is negative from get_fundamentals.derived.net_cash
- thin earnings history with only 1 quarter from get_earnings_history.summary.beat_rate

Data gaps:
- get_fundamentals: returnOnEquity missing
- get_fundamentals: earningsGrowth missing
- get_fundamentals: not applicable for this company: trailingPE, enterpriseToEbitda, pegRatio
- get_earnings_history: not applicable for this company: FY2026Q2.surprise_pct

```
  - get_earnings_history: thin history: only 1 classifiable quarter(s) (expe
cted 4)

Sources:
  - Yahoo Finance via yfinance (fundamentals)
  - Finnhub (earnings history)
  - https://finnhub.io/api/news?id=227fd0e89b1e80a476045facd0d62b89eb7f53d13
204fad2ca3e338dbe0530f1
  - https://finnhub.io/api/news?id=0b31a88a4d9a99d8224f34558b4bba91437d15d8d
7e530e0dea18236b3c95f24
  - https://finnhub.io/api/news?id=f413d05e558971fa226282f43799d3bd9121ce28b
70b314a815e0720cbaa255f

Educational analysis produced by a course project. Not investment advice.

stage errors: []
```

In [5]:
```
# The news workflow's output inside this run: one insight per analyzed
article.
# (The analysis above lists these articles under Sources; news is one input
to the thesis.)
news = run_aadx.loop["results"].get("get_news_insights", [])
print(len(news), "news insights for AADX\n")
for item in news:
    print(f'{item["published"]}  {item["category"]:<8} {item["sentiment"]:
<8} {item["summary"]}')
    print("   ", item["source_url"])
```

```
3 news insights for AADX

2026-10-13  earnings positive Applied Aerospace & Defense expects mid-teens
organic revenue growth in 2027 and 2028, supported by defense demand and bac
klog, according to RBC's assessment.
    https://finnhub.io/api/news?id=227fd0e89b1e80a476045facd0d62b89eb7f53d13
204fad2ca3e338dbe0530f1
2026-10-13  general  neutral  RBC Capital maintains an Outperform rating on
Applied Aerospace (NYSE:AADX), while lowering the price target from $24 to
$21, with analyst Ken Herbert involved in this rating change.
    https://finnhub.io/api/news?id=0b31a88a4d9a99d8224f34558b4bba91437d15d8d
7e530e0dea18236b3c95f24
2026-10-13  general  positive Jim Cramer recommends sticking with Applied Ae
rospace, favoring it alongside Nokia and BlackBerry, while questioning Ambar
ella and expressing a dislike for food stocks.
    https://finnhub.io/api/news?id=f413d05e558971fa226282f43799d3bd9121ce28b
70b314a815e0720cbaa255f
```

In [6]:
```
for bad in ("ZZZZZZ", "SPY"):
    r = run_pipeline(bad, analyze=True)
    print(bad, "->", r.loop["stop_reason"], "|", r.loop["fatal_error"])
```

```
HTTP Error 404: {"quoteSummary":{"result":null,"error":{"code":"Not Foun
d","description":"Quote not found for symbol: ZZZZZZ"}}}
HTTP Error 404: {"quoteSummary":{"result":null,"error":{"code":"Not Foun
d","description":"Quote not found for symbol: ZZZZZZ"}}}
ZZZZZZ -> fatal_error | InvalidTickerError: ZZZZZZ: no data returned; check
the symbol
```

```
HTTP Error 404: {"quoteSummary":{"result":null,"error":{"code":"Not Found","description":"No fundamentals data found for symbol: SPY"}}}
SPY -> fatal_error | NotAStockError: SPY is a ETF, not a single stock
```

1c. The same thing from a terminal

```
python -m finagent.run_agent NVDA                          # data loop only
python -m finagent.run_agent NVDA --analyze                # + the analysis
python -m finagent.run_agent NVDA --analyze --evaluate     # + judge and revision
python -m finagent.run_agent NVDA --analyze --evaluate --json   # everything as JSON
python -m finagent.run_agent NVDA --analyze --no-memory    # skip the run diary
```

2. Each module on its own

Run one stage at a time to see exactly what it produces. This is the best way to debug.

2.1 Company profile (the pre-flight check)

Decides if the ticker is a real single stock, and sets context (size, profitability, days to earnings) that shapes the plan.

In [7]:
```
from finagent.tools.company_profile import get_company_profile
get_company_profile("NVDA")
```

Out[7]:
```
Profile(ticker='NVDA', name='NVIDIA Corporation', quote_type='EQUITY', sector='Technology', industry='Semiconductors', market_cap=5170596937728.0, size_bucket='mega', is_profitable=True, is_pre_revenue=False, next_earnings_date=datetime.date(2026, 11, 17), days_to_earnings=30, warnings=[])
```

2.2 Fundamentals (valuation, margins, growth, balance sheet)

Look at `gaps`: they say *why* a value is missing (for example `negative_earnings` for a loss-maker).

In [8]:
```
from finagent.tools.fundamentals import get_fundamentals
f = get_fundamentals("AADX")
print("gaps:", f["gaps"])
f["valuation"], f["profitability"]
```

```
gaps: [{'field': 'trailingPE', 'reason': 'negative_earnings'}, {'field': 'enterpriseToEbitda', 'reason': 'not_meaningful'}, {'field': 'pegRatio', 'reason': 'negative_earnings'}, {'field': 'returnOnEquity', 'reason': 'missing'}, {'field': 'earningsGrowth', 'reason': 'missing'}]
```

Out[8]:
```
({'trailingPE': None,
  'forwardPE': 28.603773,
  'priceToBook': 2.3677633,
  'priceToSalesTrailing12Months': 3.403514,
  'enterpriseToEbitda': None,
  'pegRatio': None},
 {'trailingEps': -1.42,
  'grossMargins': 0.25565,
  'operatingMargins': -0.57514,
  'profitMargins': -0.30236998,
  'returnOnEquity': None})
```

2.3 Earnings history (beat or miss vs analyst estimates)

In [9]:
```
from finagent.tools.earnings_history import get_earnings_history
h = get_earnings_history("NVDA")
print(h["summary"])
for q in h["quarters"]:
    print(q)
```

```
{'n': 4, 'beats': 4, 'in_line': 0, 'misses': 0, 'beat_rate': 1.0, 'avg_surpr
ise_pct': 3.442516816021538, 'latest_surprise_pct': 3.8159371492704985}
{'year': 2027, 'quarter': 2, 'estimate': 2.1384, 'actual': 2.22, 'surprise_p
ct': 3.8159371492704985, 'result': 'beat'}
{'year': 2027, 'quarter': 1, 'estimate': 1.7922, 'actual': 1.87, 'surprise_p
ct': 4.341033366811745, 'result': 'beat'}
{'year': 2026, 'quarter': 4, 'estimate': 1.5634, 'actual': 1.62, 'surprise_p
ct': 3.6203146987335426, 'result': 'beat'}
{'year': 2026, 'quarter': 3, 'estimate': 1.2746, 'actual': 1.3, 'surprise_pc
t': 1.9927820492703665, 'result': 'beat'}
```

2.4 Planner + research loop

The **model decides** which tools to call, in what order. `trace` shows every step; step 0 is the pre-flight check.

In [10]:
```
from finagent.agent.loop import run_research_loop
loop = run_research_loop("NVDA")
print("plan emphasis:", loop["plan"]["priority_note"])
for step in loop["plan"]["steps"]:
    print(" -", step["tool"], "|", step["reason"][:90])
print()
for t in loop["trace"]:
    print(t["step"], t["tool"], "ok" if t["ok"] else "ERROR " +
str(t["error"]))
```

```
plan emphasis: This plan emphasizes a comprehensive analysis of NVIDIA's fin
ancial health and market sentiment ahead of its upcoming earnings report.
 - get_fundamentals | As a mega-cap technology company, understanding NVIDI
A's key financial ratios will provide
 - get_earnings_history | With upcoming earnings in 30 days, analyzing the l
ast four quarters of earnings versus ana
 - get_news_insights | Recent news can impact investor sentiment and stock p
erformance, making it crucial to unde

0 get_company_profile ok
0 plan_research ok
1 get_fundamentals ok
1 get_earnings_history ok
1 get_news_insights ok
```

2.5 Reflection

Code decides here, not the model: it retries a failed tool once if a retry can help, and reports what remains.

In [11]:
```
from finagent.agent.reflection import reflect
loop2, reflection = reflect(loop)
reflection["summary"], reflection["retried"]
```

Out[11]:
```
('Nothing to retry; 0 gaps remain (retrying cannot fix them).', [])
```

2.6 Signals (computed by Python)

Each is one checkable judgment with its source field. The model must account for every one.

In [12]:
```
from finagent.agent.signals import compute_signals
for s in compute_signals(loop2["results"]):
    print(f"{s['name']:<24} {s['direction']:<9} {s['value']!s:<14}
{s['why'][:70]}")
```

```
forward_vs_trailing_pe   positive  13.454941      forward P/E 13.5 is below
trailing 27.1: analysts expect earnings to g
profit_margin            positive  0.63663        net profit margin is healt
hy (10% or more)
revenue_growth           positive  1.059          sales are growing 10% a ye
ar or faster
net_cash                 positive  23608999936.0  cash exceeds debt (a net c
ash position)
fcf_yield                neutral   0.024563121343549824 free cash flow is po
sitive but under 3% of market value
liquidity                positive  4.589          short-term assets comforta
bly cover short-term bills
trend_vs_200d            positive  214.13         price 214.13 is above its
200-day average 202.97 (the long-term trend)
analyst_target           positive  328.71695      analyst average target 32
8.72 is +54% from the price (59 analysts; an
earnings_record          neutral   1.0            beat estimates in 4 of 4 q
uarters; beating estimates is the norm (abou
```

2.7 The news pipeline, station by station

`Ingest -> Preprocess -> Classify -> Route -> Extract -> Summarize`.

Run the stations separately to see what each one removes.

In [13]:
```python
from finagent.workflows.news_ingest import collect_articles
batch = collect_articles("NVDA", company_name="NVIDIA")
print("kept:", len(batch.articles), "| window days:", batch.window_days, "|
widened:", batch.widened)
for a in batch.articles[:5]:
    print(a.published.date(), "|", a.headline[:90])
```

```
kept: 20 | window days: 14 | widened: False
2026-10-19 | This Is AMD's Biggest Hurdle in an AI Sell-Off (and No, It's No
t Nvidia)
2026-10-19 | Here’s What a $10,000 Investment in Nvidia Stock Will Be Worth
by 2030
2026-10-18 | The Bond Market Is Sending a Warning to Nvidia Shareholders
2026-10-18 | NVIDIA And 2 Other Fast Growing Stocks To Own
2026-10-18 | The next AI race will be won at the credit desk
```

In [14]:
```python
from finagent.agent.loop import make_client
from finagent.workflows.news_classify import classify_articles,
make_llm_classifier

client = make_client()
classified = classify_articles(batch.articles, "NVIDIA",
make_llm_classifier(client))
print("kept:", len(classified.items), "| irrelevant:",
classified.irrelevant,
      "| failed:", classified.failed, "| unchecked:", classified.unchecked)
for article, label in classified.items:
    print(f"{label:<9} {article.headline[:80]}")
```

```
kept: 8 | irrelevant: 8 | failed: 0 | unchecked: 4
market    Here’s What a $10,000 Investment in Nvidia Stock Will Be Worth by
2030
market    The Bond Market Is Sending a Warning to Nvidia Shareholders
general   The next AI race will be won at the credit desk
general   What a New Nvidia-Backed AI Platform Means for Honeywell Stock
market    History Says Nvidia Stock Does This in November. Here's the 1 Thin
g That Could B
market    Why Did NVIDIA (NVDA) Rise Only 19% While Its Suppliers Did Far Be
tter?
general   Elon Musk Says Jensen Huang 'Rocks' as SpaceX, Nvidia Team Up on A
I Data Centers
general   How Nvidia Could Make $150 Billion Look Small
```

In [15]:
```python
from finagent.workflows.news_specialists import build_insights,
make_llm_analyzer

insights = build_insights(classified, "NVIDIA", make_llm_analyzer(client))
for i in insights.insights:
    print(i["published"], i["category"], i["sentiment"], "|", i["summary"])
```

```
2026-10-19 market positive | Emerging growth opportunities in the AI ecosyst
em are anticipated, and Nvidia stock is projected to increase in value by 20
30.
2026-10-18 market negative | Bond investors are showing less confidence in N
vidia, which serves as a warning to Nvidia shareholders.
2026-10-18 general positive | Nvidia is launching a $500 billion financing i
nitiative as the AI race shifts its focus to capital markets, indicating tha
t engineering is no longer the only critical factor in this competition.
2026-10-18 general positive | Honeywell has introduced the Forge Cognition p
latform, which is backed by Nvidia.
2026-10-18 market positive | Nvidia stock has historically produced an avera
ge return of 13% in November this decade, although there is a suggestion tha
t one factor could disrupt this trend.
2026-10-18 market neutral | NVIDIA closed at $217.23 on October 15 and has r
eturned 9.52% over the last six months, while also being a key player in des
igning processors for artificial intelligence.
2026-10-18 general positive | Elon Musk praised Nvidia CEO Jensen Huang, hig
hlighting their collaboration on AI computers for space, which aims to devel
op AI data centers in that environment.
2026-10-18 general positive | Nvidia has announced its largest share repurch
ase authorization in market history, with the company currently valued at $5
trillion.
```

One article through the specialist chain. `build_insights` does this for every article. Here it is done by hand for one, so each step is visible: the label is **routed** to a specialist, **Extract** pulls short facts and a sentiment, and **Summarize** writes the summary from those facts only. Python validates each reply (labels, sentiment, and that every number appears in the article).

In [16]:
```python
from finagent import config
from finagent.workflows.news_classify import build_user_message
from finagent.workflows.news_specialists import (
    EXTRACT_PROMPT, MAX_FACTS, SUMMARIZE_PROMPT, parse_extract,
parse_summary, route,
```

```
)

def ask(system, user):
    reply = client.chat.completions.create(
        model=config.MODEL, temperature=config.TEMPERATURE, max_tokens=300,
        messages=[{"role": "system", "content": system}, {"role": "user",
"content": user}])
    return reply.choices[0].message.content or ""

article, label = classified.items[1]
specialist = route(label)
print("Article   :", article.headline)
print("Classify  :", label, "->", "routed to", specialist.name)

facts, sentiment = parse_extract(
    ask(EXTRACT_PROMPT.format(specialist=specialist.name, subject="NVIDIA",
                              focus=specialist.focus, max_facts=MAX_FACTS),
        build_user_message(article)), article)
print("Extract   : facts =", facts, "| sentiment =", sentiment)

summary = parse_summary(
    ask(SUMMARIZE_PROMPT.format(specialist=specialist.name,
subject="NVIDIA"),
        "Key facts:\n" + "\n".join(f"- {f}" for f in facts)), article)
print("Summarize :", summary)
```

```
Article   : The Bond Market Is Sending a Warning to Nvidia Shareholders
Classify  : market -> routed to Market specialist
Extract   : facts = ['Bond investors are showing less confidence in Nvidi
a.', 'This is a warning to Nvidia shareholders.'] | sentiment = negative
Summarize : Bond investors are showing less confidence in Nvidia, which serv
es as a warning to Nvidia shareholders.
```

The news workflow's final output is one `NewsInsight` per article (category, summary, sentiment, source URL, date). There is no single combined news summary by design: the agent's Synthesis step (2.8) reads these insights together with the fundamentals and earnings. A quick roll-up, computed by Python:

In [17]:
```
from collections import Counter

print("kept after classify   :", len(classified.items), "| dropped as
irrelevant:", classified.irrelevant)
print("insights produced     :", len(insights.insights), "| failed a
check:", insights.failed)
print("routed to specialist  :", dict(Counter(route(i["category"]).name for
i in insights.insights)))
print("sentiment mix         :", dict(Counter(i["sentiment"] for i in
insights.insights)))
print("newest article        :", max(i["published"] for i in
insights.insights))
```

```
kept after classify   : 8 | dropped as irrelevant: 8
insights produced     : 8 | failed a check: 0
routed to specialist  : {'Market specialist': 4, 'News specialist': 4}
sentiment mix         : {'positive': 6, 'negative': 1, 'neutral': 1}
newest article        : 2026-10-19
```

In [18]:
```python
# The same chain as one tool call (what the agent actually uses):
from finagent.tools.news import get_news_insights
tool_result = get_news_insights("AADX")
print(len(tool_result), "insights; first one:")
print(tool_result[0] if tool_result else "none")
```

```
3 insights; first one:
{'category': 'earnings', 'summary': 'Applied Aerospace & Defense expects mid
-teens organic revenue growth in 2027 and 2028, with RBC supporting this out
look based on defense demand and backlog.', 'sentiment': 'positive', 'source
_url': 'https://finnhub.io/api/news?id=227fd0e89b1e80a476045facd0d62b89eb7f5
3d13204fad2ca3e338dbe0530f1', 'published': '2026-10-13'}
```

2.8 Synthesis (the thesis)

The model writes the thesis. **Python** builds the gaps and sources, checks every evidence number, and caps the confidence.

News is one input here: the model reads the insights from 2.7 together with fundamentals and earnings.

In [19]:
```python
from finagent.agent.synthesis import analyze_run
outcome = analyze_run(loop2, reflection=reflection)
print(outcome.analysis["stance"], outcome.analysis["confidence"])
print(outcome.analysis["thesis"])
print("notes from Python:", outcome.notes)
print("fell back to rule-based?", outcome.fallback_reason)
```

```
bullish high
NVIDIA Corporation demonstrates strong financial health and growth potentia
l, supported by a solid profitability profile and a positive outlook from an
alysts. The company has a net cash position of $23.6 billion, indicating a s
trong balance sheet, and its revenue growth rate of 1.059 suggests robust sa
les performance. Additionally, the forward P/E ratio of 13.45 being lower th
an the trailing P/E of 27.07 indicates expected earnings growth, further bol
stered by a healthy profit margin of 63.66%.
notes from Python: []
fell back to rule-based? None
```

2.9 Memory (offline, uses a temp file)

Append-only diary of past analyses; `diff_runs` says what changed since the last run. Entries older than 30 days are ignored when reading but never deleted.

In [20]:
```python
import tempfile
from pathlib import Path
from datetime import UTC, datetime, timedelta
```

```
from finagent.memory import record_from_outcome, save_run, load_prior,
diff_runs, format_diff

tmp = Path(tempfile.mkdtemp()) / "memory.jsonl"
week_ago = datetime.now(UTC) - timedelta(days=7)

old = record_from_outcome(outcome, now=week_ago)
old["stance"] = "neutral"                       # pretend last week's view
was different
save_run(old, tmp)

new = record_from_outcome(outcome)
prior = load_prior(new["ticker"], tmp)
print("\n".join(format_diff(diff_runs(prior, new))))
```

```
Compared with the run 7 day(s) ago:
  stance: neutral -> bullish
```

2.10 Memory across two live runs

The same pipeline with `memory_path` set. The first run saves; the second loads it and prints the diff. Small changes (a moved price, a shifted signal) are expected between runs.

In [21]:
```
# Live: run the same ticker twice with memory on (a temp file, so data/ is
untouched).
# The second run finds the first and reports what changed.
import tempfile
from pathlib import Path
from finagent.memory import format_diff

mem_file = Path(tempfile.mkdtemp()) / "memory.jsonl"
first = run_pipeline("NVDA", analyze=True, memory_path=mem_file)
second = run_pipeline("NVDA", analyze=True, memory_path=mem_file)
print("first run : saved =", first.saved, "| prior =", first.prior is not
None)
print("second run: saved =", second.saved, "| prior found =", second.prior
is not None)
print("\n".join(format_diff(second.changes)) if second.changes else "no
diff")
print("lines in memory file:", len(mem_file.read_text().splitlines()))
```

```
first run : saved = True | prior = False
second run: saved = True | prior found = True
Compared with the run 0 day(s) ago:
  no change in stance, confidence, signals or gaps
lines in memory file: 2
```

Agent functions recap

Function	Who decides	Code	Shown in
Planning	model	`plan_research`	1a, 2.4

Function	Who decides	Code	Shown in
Dynamic tool use	model picks, Python validates and runs	`run_research_loop`, `dispatch`	1a, 2.4
Self-reflection	Python	`reflect`	2.5
Learns across runs	Python	`load_prior`, `save_run`, `diff_runs`	2.9, 2.10

Memory is an append-only file (`data/memory.jsonl`). Nothing is deleted; records older than 30 days are ignored when reading. The next run reports what changed in stance, confidence, signals and data gaps. In this version the prior record appears in the report and is not yet fed back into the model's prompt.

Evaluation and Iteration

Part 3 covers how output quality is checked and improved: the evaluator-optimizer loop (judge scores, Python decides, revise with feedback), the Python safety checks that run on every analysis, and the known limits.

3. Must-see behaviors (mostly offline)

3.1 The tests

Every module has offline tests (no model, no network).

In [22]:
```
import sys
!PYTHONPATH=src {sys.executable} -m pytest -q
```

```
.................
..................................................
.. [ 16%]
.........................................
..............
............ [ 33%]
........................
.......................................... [ 50%]
.....
.......
....................................
..................... [ 67%]
....................................
................................... [ 83%]
..................................
..................................    [100%]
429 passed in 1.90s
```

3.2 Fail loudly, not quietly

A failed data fetch **raises**, so the trace shows `ok=False` and reflection can retry. Returning an empty result would look like success.

In [23]:
```
from types import SimpleNamespace
from finagent.tools.earnings_history import get_earnings_history

class BrokenClient:
    def company_earnings(self, *a, **k):
        raise ConnectionError("network down")

try:
    get_earnings_history("NVDA", client=BrokenClient())
except RuntimeError as e:
    print("raised:", e)
```

```
raised: Finnhub call failed: ConnectionError: network down
```

3.3 Safety check: the model cannot invent numbers in a news summary

Every number in a summary must appear in the article, or the item is rejected.

In [24]:
```
from datetime import UTC, datetime
from finagent.workflows.news_ingest import Article
from finagent.workflows.news_specialists import check_numbers_grounded,
SpecialistError

article = Article(headline="Acme raises guidance", summary="Revenue
guidance now 5 billion dollars.",
                  url="https://example.com", source="demo",
published=datetime.now(UTC))
check_numbers_grounded("Guidance is now 5 billion dollars.", article)   #
passes
try:
    check_numbers_grounded("Guidance is now 7 billion dollars.", article)
# 7 is invented
except SpecialistError as e:
    print("rejected:", e)
```

```
rejected: numbers not in the source: ['7']
```

3.4 Stale news is reported as a data gap

In [25]:
```
from datetime import date, timedelta
from finagent.agent.synthesis import build_data_gaps
from finagent.samples import SAMPLE_LOOP_RESULT

old_news = [{"published": (date.today() - timedelta(days=40)).isoformat(),
             "category": "general", "summary": "x", "sentiment": "neutral",
"source_url": "u"}]
```

```
stale = {**SAMPLE_LOOP_RESULT, "results": {**SAMPLE_LOOP_RESULT["results"],
"get_news_insights": old_news}}
[g for g in build_data_gaps(stale) if "news" in g]
```

Out[25]:
```
['get_news_insights: stale, the newest article is 41 days old']
```

Evaluation and iteration: design comments

Goal: generate an analysis, evaluate its quality, refine it using the feedback (the evaluator-optimizer pattern).

How it works (`src/finagent/workflows/evaluator_optimizer.py`):

1. **Generate:** synthesis writes a draft (stance, confidence, thesis, evidence, risks).
2. **Evaluate:** a separate judge call scores six criteria from 1 to 5 against the *raw tool data*: `evidence_support`, `numeric_accuracy`, `stance_consistency`, `risk_coverage`, `confidence_calibration`, `source_quality`. It also lists any critical issues.
3. **Decide (Python, not the model):** pass only if the mean is at least 4.0, no score is below 3, and there is no critical issue. A rule in code cannot be talked into a pass.
4. **Refine:** a failing draft goes back through synthesis with the judge's notes. The revision passes the same Python guards as a first draft (verifiable evidence, confidence ceiling), so it cannot invent evidence to raise its score.
5. **Stop:** on a pass, after at most **2 revisions**, or if the judge or reviser fails. The best draft is kept, and a draft that never passes is marked *not passed*.

How iteration was done during development: compare the first and revised drafts on the same collected data, record scores and critical issues, and review by hand whether a revision addressed the feedback before changing prompts or thresholds. The 429 offline tests pin the rule, the revision cap and every failure path.

Honest limits: by default the same model writes and judges (`config.JUDGE_MODEL`), which can be lenient, and the judge has not been calibrated against human ratings. See 3.7.

3.5 The evaluator-optimizer loop, with a scripted judge (no API)

The **judge scores, Python decides.** A scripted judge lets you see the loop's behavior exactly: a failing draft is revised, the loop stops after at most 2 revisions, and the best draft is kept.

In [26]:
```
from finagent.agent.synthesis import SynthesisOutcome
from finagent.samples import SAMPLE_ANALYSIS
from finagent.workflows.evaluation_rules import RUBRIC_CRITERIA,
decide_verdict
from finagent.workflows.evaluator_optimizer import Judgment,
evaluate_and_optimize
```

```
def judgment(value, critical=None):
    scores = [{"criterion": c, "score": value, "comment": f"{c} needs
work"} for c in RUBRIC_CRITERIA]
    return Judgment(scores, critical_issues=critical or [])

draft = SynthesisOutcome({**SAMPLE_ANALYSIS, "stance": "neutral"}, None,
[], [])
better = SynthesisOutcome({**SAMPLE_ANALYSIS, "stance": "bullish"}, None,
[], [])

judgments = iter([judgment(2), judgment(5)])           # draft fails,
revision passes
result = evaluate_and_optimize(draft, {}, lambda a, r, s: next(judgments),
lambda fb, prev: better)
print("passed:", result.passed, "| revisions:", result.revisions, "|",
result.stopped_because)
print("final stance:", result.final.analysis["stance"])
```

```
passed: True | revisions: 1 | the revision 1 passed
final stance: bullish
```

3.5b The verdict rule in isolation

In [27]:
```
# The verdict rule: mean >= 4.0, no score below 3, no critical issue.
def check(values, critical=None):
    scores = [{"criterion": c, "score": v, "comment": ""} for c, v in
zip(RUBRIC_CRITERIA, values)]
    return decide_verdict(scores, critical_issues=critical)

print(check([5, 5, 5, 5, 5, 5]))                          # pass
print(check([5, 5, 5, 5, 5, 2]))                          # average is
high, but one score is below 3
print(check([5, 5, 5, 5, 5, 5], ["number invented"]))     # a critical
issue blocks a pass
```

```
('pass', 5.0)
('revise', 4.5)
('revise', 5.0)
```

3.6 AI disclosure

AI (Claude, by Anthropic) was used for brainstorming and iterating on the design, drafting and formatting code and documentation. The team critically reviewed, tested and verified the output end to end.

3.7 Known limits (worth stating in the write-up)

- **Same model writes and judges** by default (`config.JUDGE_MODEL`), which can make the judge lenient on its own writing.
- The judge is **not calibrated against human ratings** yet.

- The number check catches **invented numbers** in evidence items and news summaries. It does not check numbers written in the thesis prose, and it cannot catch invented non-numeric claims. Example in this notebook: in the 1a output the tool field `revenueGrowth` is 1.059 (Python reads it as 105.9% growth), but the thesis text says 5.9%, and the same-model judge still scored numeric accuracy 5.
- Finnhub's free news feed is **capped and noisy** (roundups tagged with the ticker), so articles are ranked by company mentions before the cap.
- News summaries skew **positive** (press releases), so news sentiment is weak evidence and is not part of the scorecard.
- Free-tier limits: Finnhub 60 calls/min, Alpha Vantage 25/day, NewsAPI ~100/day; yfinance is unofficial.

4. Summary

Requirement	Code entry point	Demonstrated in
Agent: planning	`plan_research`	1a, 2.4
Agent: dynamic tool use	`run_research_loop` , `dispatch`	1a, 2.4, 2.2, 2.3
Agent: self-reflection	`reflect` ; the evaluator's judge	2.5, 1a, 3.5
Agent: learns across runs	`load_prior` , `save_run` , `diff_runs`	2.9, 2.10
Workflow: prompt chaining	`collect_articles` → `classify_articles` → `build_insights` (inside `get_news_insights`)	2.7
Workflow: routing	`route`	2.7
Workflow: evaluator-optimizer	`evaluate_and_optimize`	1a (live), 3.5 (scripted judge), 3.5b (verdict rule)

The design principle in one sentence: **the model proposes, Python checks.** The model chooses tools and writes the thesis; code builds the signals, gaps and sources, verifies the numbers, caps the confidence, decides when to retry and decides pass or revise.

Repository, setup steps, architecture diagrams and the decision log: https://github.com/sudhakaran-srinivasan/multi-agent-financial-analyst